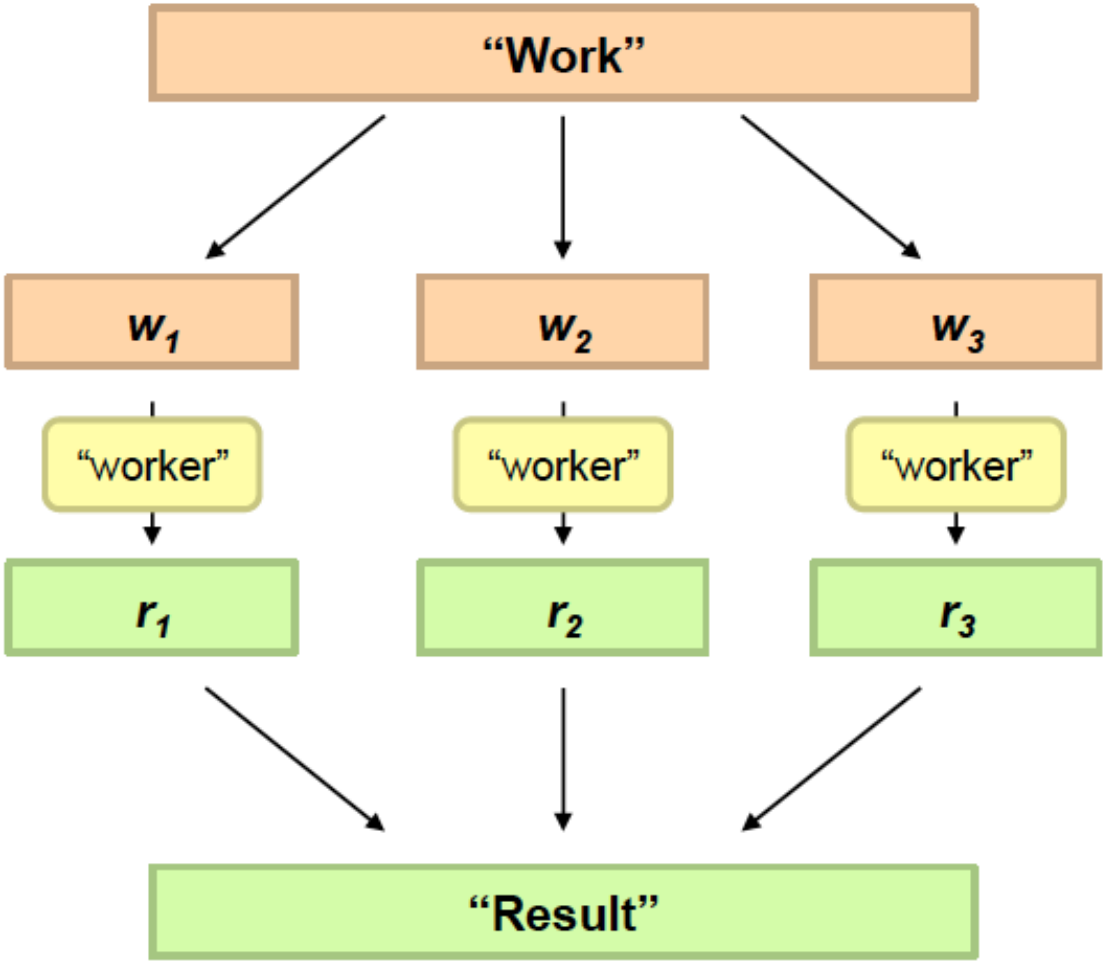
MapReduce

Philosophy to Scale for Big Data?

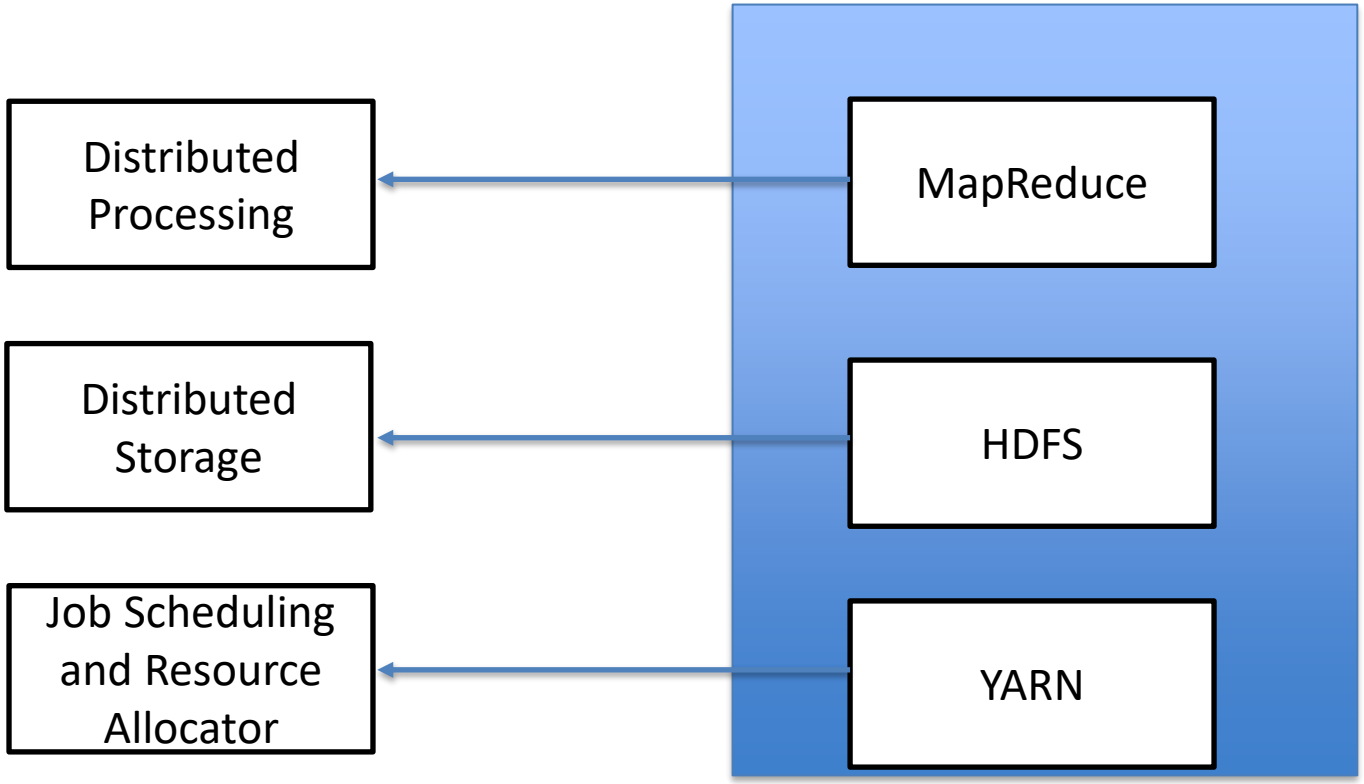
Divide and Conquer



Divide Work

Combine Results

Hadoop Core Components



MapReduce: A Real World Analogy

Coins Deposit



?



MapReduce: A Real World Analogy

Coins Deposit







Coins Counting Machine

MapReduce: A Real World Analogy

Coins Deposit



Mapper: Categorize coins by their face values

Reducer: Count the coins in each face value ***in parallel***

MapReduce

- MapReduce is a programming model used for efficient processing in parallel over large data-sets in a distributed manner.

HDFS and MapReduce

Hadoop Distributed File System (HDFS):

- **Storage System:** HDFS is a distributed file system designed to store and manage large volumes of data across multiple machines.

- **Data Replication:** HDFS divides large files into smaller blocks (typically 128 MB or 256 MB) and replicates these blocks across multiple nodes in a cluster to ensure fault tolerance.

- **Data Accessibility:** It provides a reliable and fault-tolerant storage layer, allowing data to be stored persistently and accessed by various applications running on a Hadoop cluster.

HDFS and MapReduce

MapReduce:

- **Processing Framework:** MapReduce is a programming model and processing engine used for parallel processing of large datasets.

- **Parallel Execution:** It processes data in parallel across a distributed cluster of computers, breaking tasks into **two main phases: the "Map" phase** for parallel processing, and the **"Reduce" phase** for aggregating results.

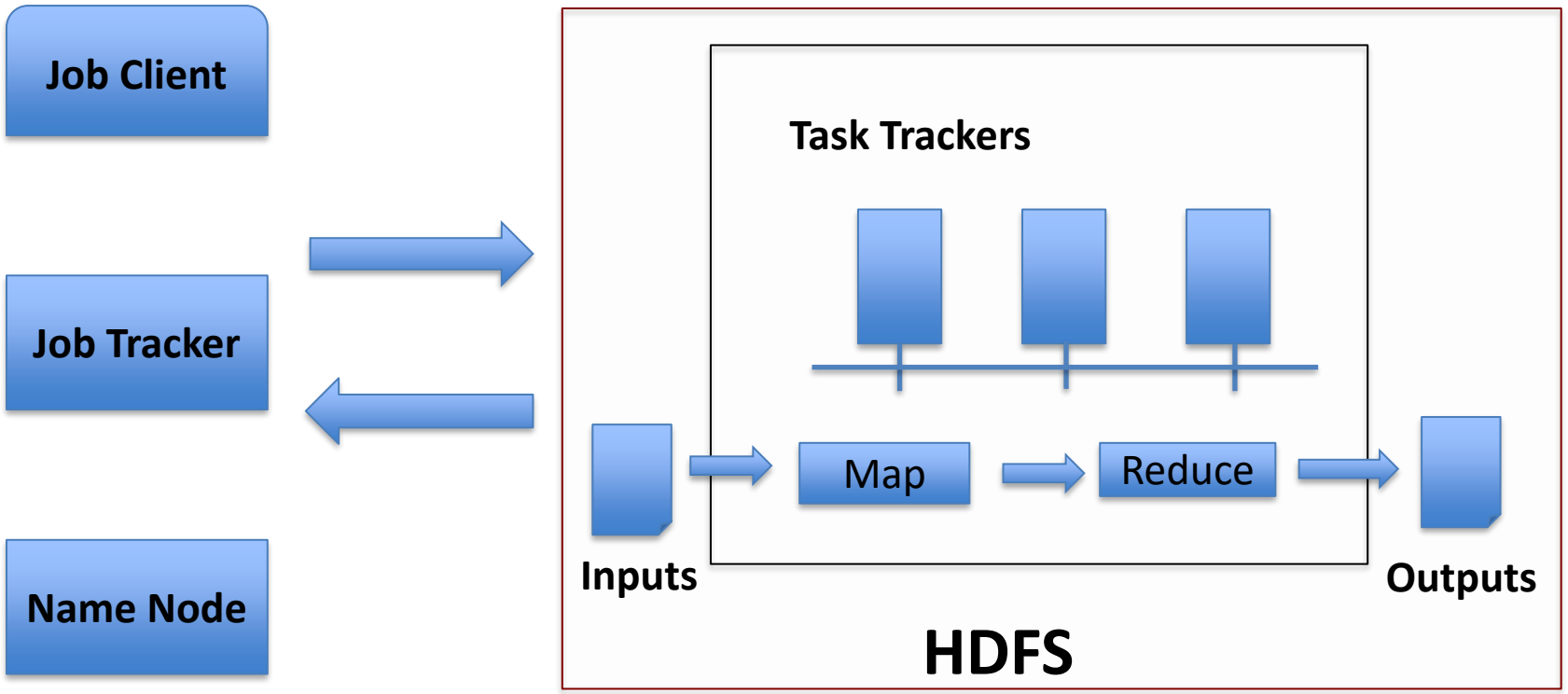
- **Scalability:** MapReduce enables scalable and efficient processing of vast amounts of data by distributing computation across multiple nodes in a Hadoop cluster.

- **Programming Model:** Developers write MapReduce programs using the Map and Reduce functions, allowing for easy parallelization of tasks.

Differences

- **Purpose:** HDFS is primarily a distributed file storage system, while MapReduce is a data processing paradigm.
- **Functionality:** HDFS is responsible for storing and managing data, providing fault tolerance and scalability. MapReduce is focused on processing and analyzing the stored data in a parallel and distributed manner.
- **Components:** HDFS is the storage layer, while MapReduce is the processing layer built on top of HDFS. They often work together within the Hadoop ecosystem.
- **Use Cases:** HDFS is used for storing large datasets, and MapReduce is used for processing and analyzing those datasets.

MapReduce Architecture: Master-Slaves



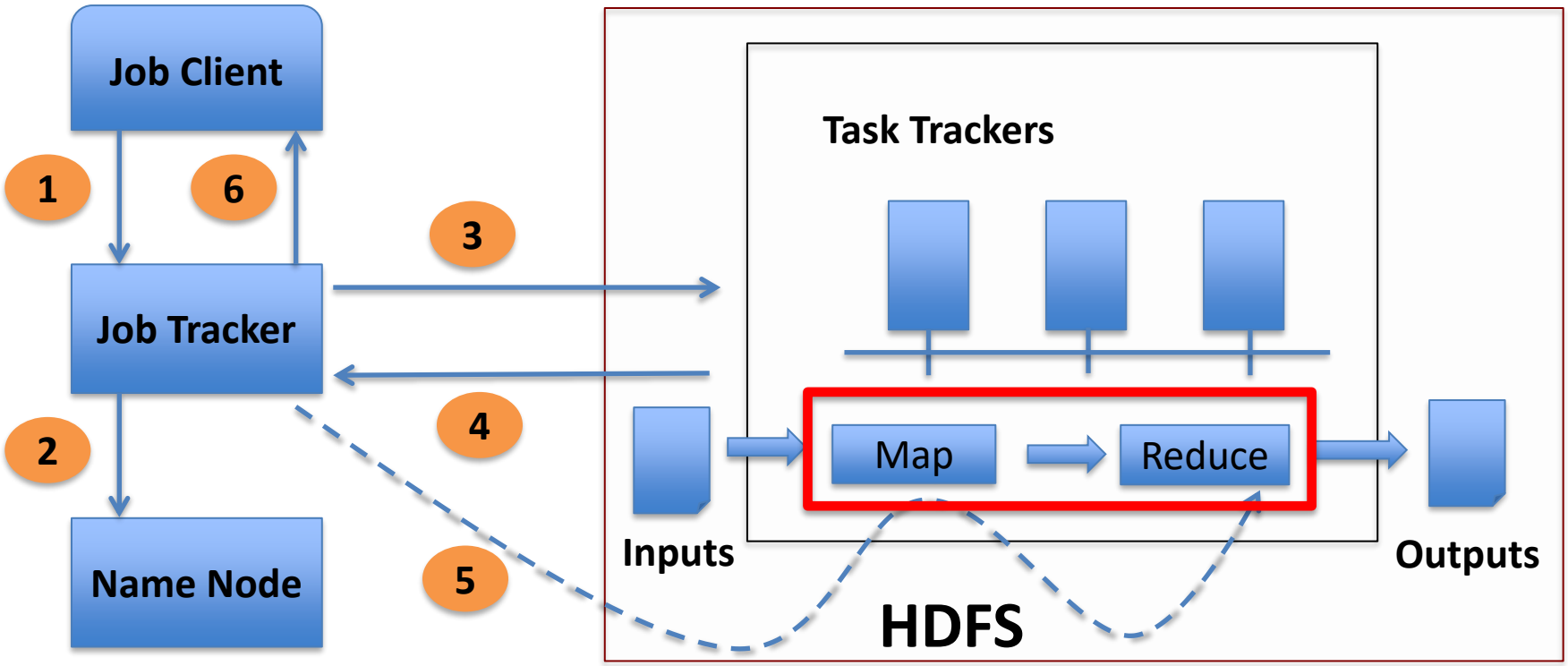
Job Client: Submit Jobs

Task Tracker: Execute Jobs

Job Tracker: Coordinate Jobs
(Scheduling, Phase Coordination, etc.)

Idea: Bring Computation to Data!

MapReduce Architecture: Workflow



1. Client submits job to Job Tracker
2. Job Tracker communicates to NN to find data it needs
3. Job Tracker creates execution plan and submits work to Task Trackers

4. Task trackers do the job and report progress/status to Job Tracker
5. Job Tracker manages task phases
6. Job Tracker finishes the job and updates status

Map-Reduce Programming Model

Example Problem: Counting Words

- We have a huge text document and count the number of times each distinct word appears in the file

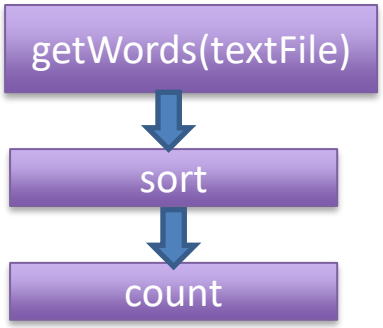
How you solve this using a single machine?

Word Count

- Case 1: File too large for memory, but all <word, count> pairs fit in memory
  - You can create a big string array OR you can create a hash table

- Case 2: All <word, count> pairs do not fit in memory, but fit into disk
  - A possible approach (write computer programs/functions for each step)

1. **Break the text document into sequence of words**
2. **Sort the words: This will bring same words together**
3. **Count the frequencies in a single pass**



- Case 2 captures the essence of Map-Reduce

MapReduce Paradigm

- Implement two functions:
  - **Map** (k1,v1) -> list (k2, v2)
  - **Reduce**(k2, list(v2)) -> list (v3)
- Framework handles everything else
- Value with the **same key** go to the same reducer

Phases

1**. Map Phase**

- Input data is split and given to Map tasks.
- Each Map task processes its local split and produces intermediate results in the form of key–value pairs.

2. Shuffle & Sort (Between Map and Reduce)

- Intermediate key–value pairs from the Map phase are grouped by key.
- Sorting ensures ordered keys before passing them to the Reducers.

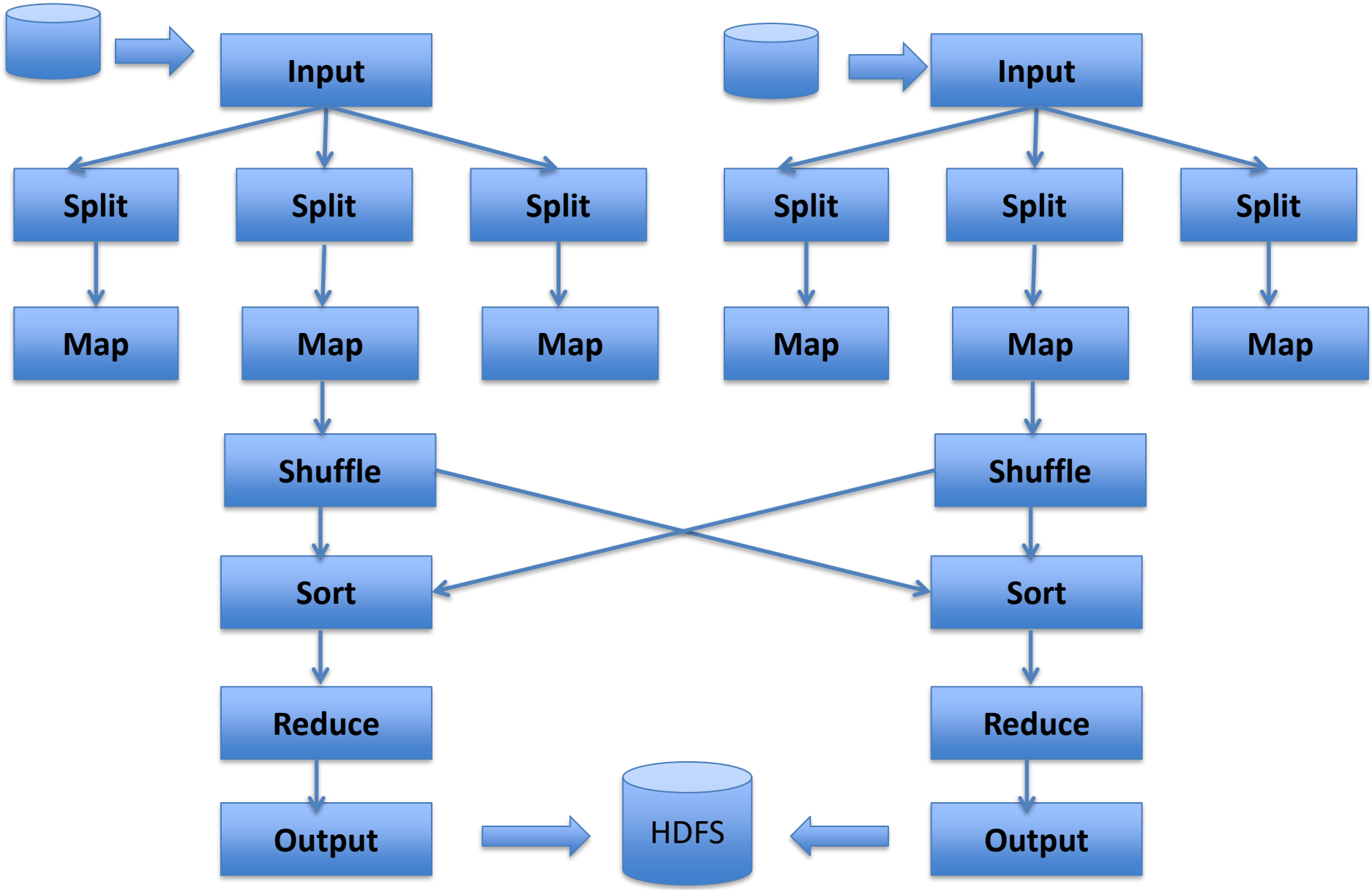
3. Reduce Phase

- Reducers take grouped keys and values from Shuffle & Sort.
- They aggregate them to produce the final output data, which is written back to HDFS.

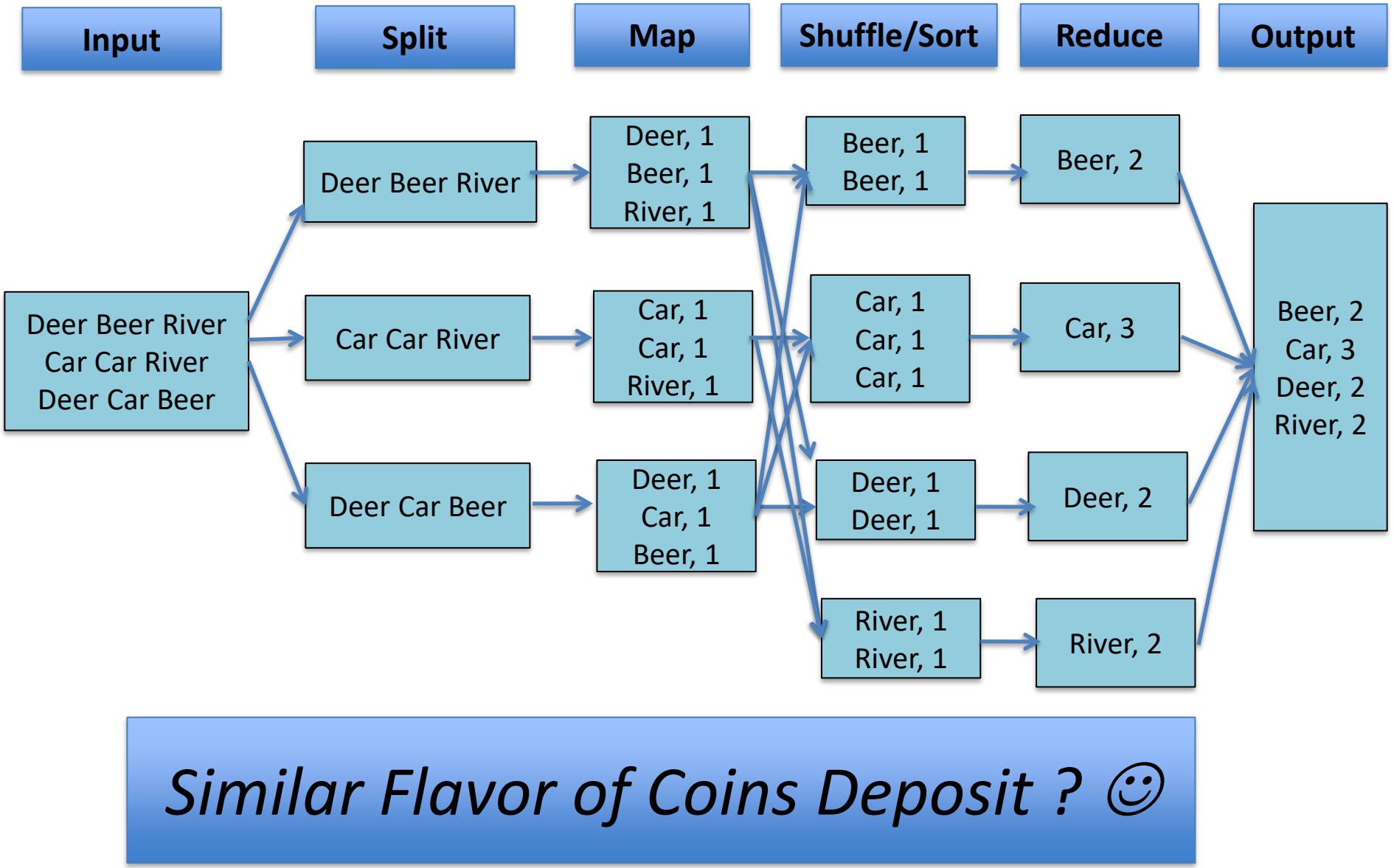
Execution Flow of MapReduce Architecture

1. Client submits the job includes Mapper & Reducer logic, input path and output path.
2. Job Tracker accepts the job, splits it into tasks and assigns them to Task Trackers based on data locality.
3. Task Trackers execute Map Tasks on input splits stored locally.
4. Intermediate results are shuffled and sorted.
5. Reduce Tasks process grouped results and generate the final output.
6. Final output is written back to HDFS.

MapReduce Internal

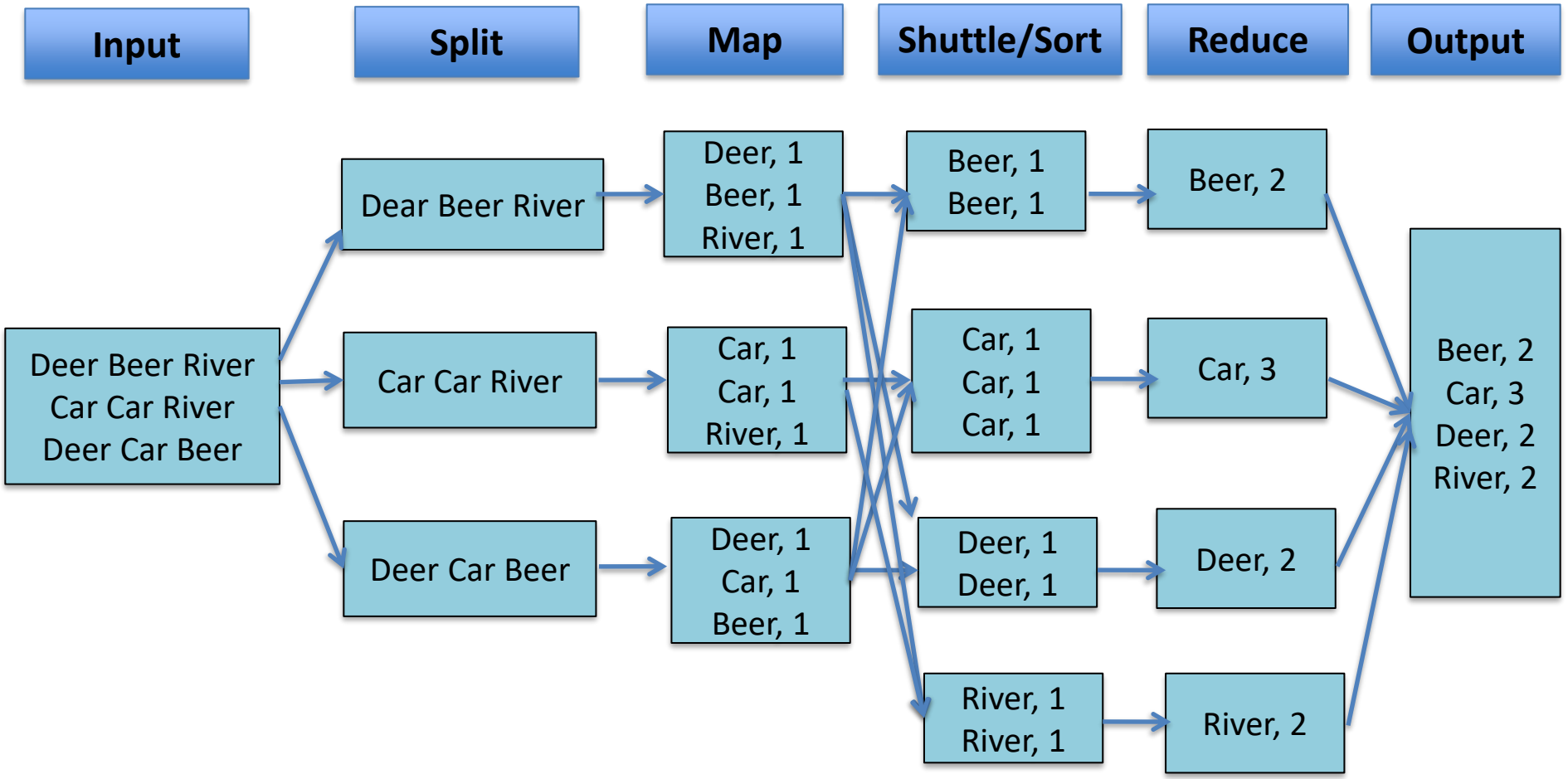


MapReduce Example: Word Count



Similar Flavor of Coins Deposit ? ☺

MapReduce Example: Word Count



Q: What are the Key and Value Pairs of Map and Reduce?

Map: Key=word, Value=1

Reduce: Key=word, Value=aggregated count

Mapper and Reducer of Word Count

- Map(key, value){

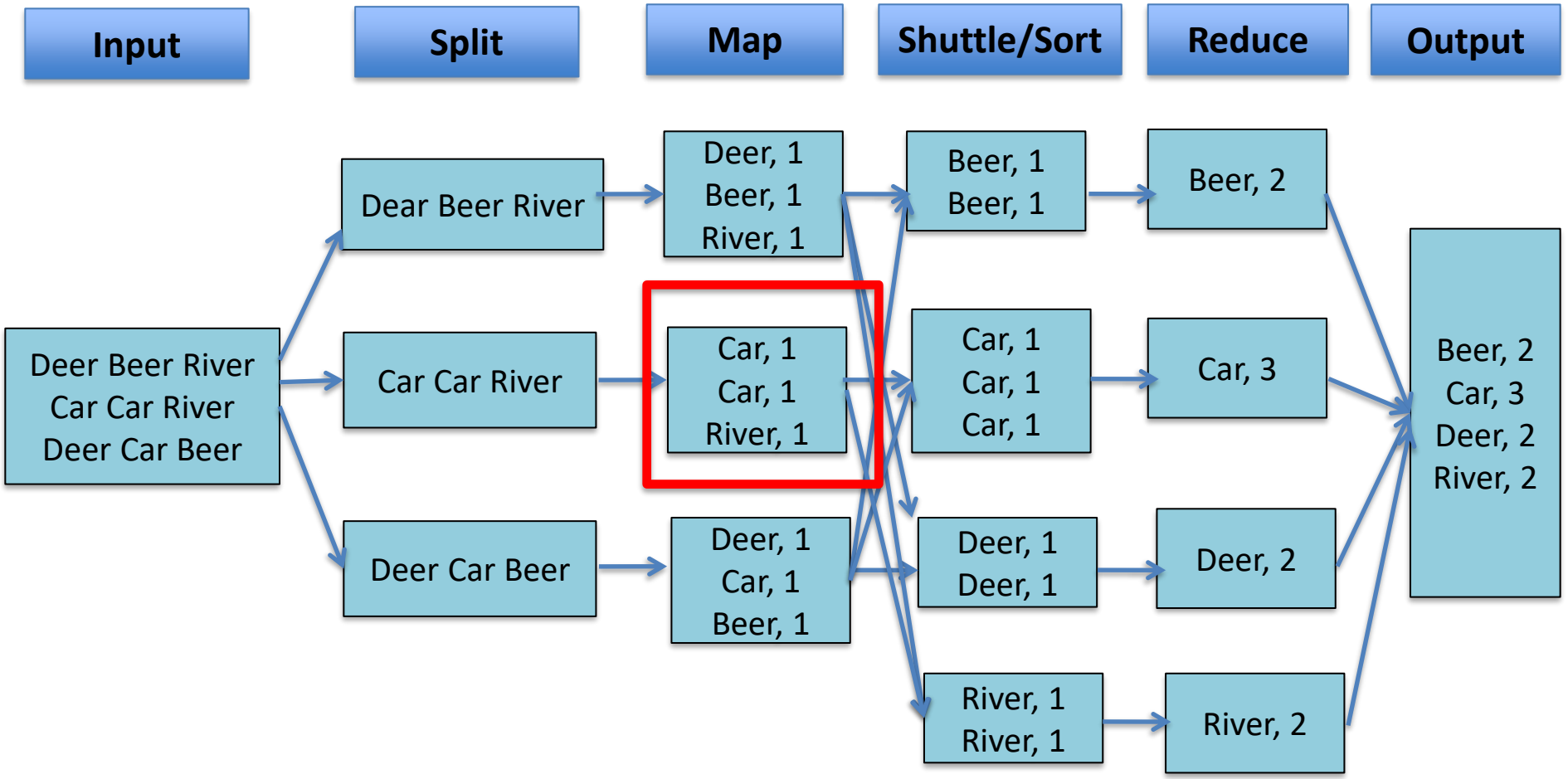
```
// key: line number
// value: words in a line
for each word w in value:
        Emit(w, "1");}
```

- Reduce(key, list of values){

```
// key: a word
// list of values: a list of counts
int result = 0;
for each v in values:
        result += ParseInt(v);
Emit(key, result);}
```

Combiner is the same as Reducer

MapReduce Example: Word Count

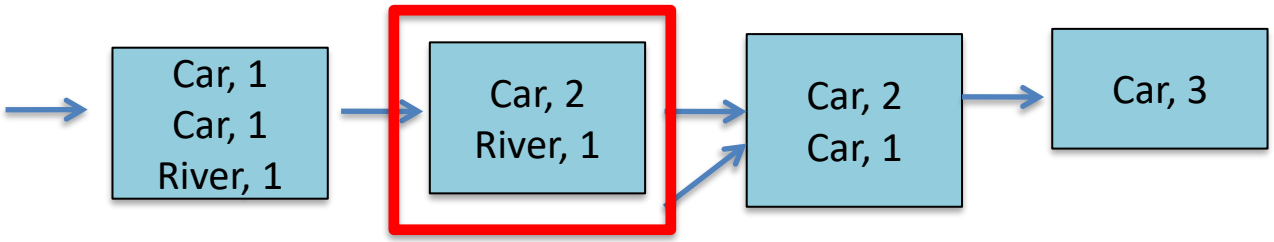


Q: Do you see any place we can improve the efficiency?

Local aggregation at mapper will be able to improve MapReduce efficiency.

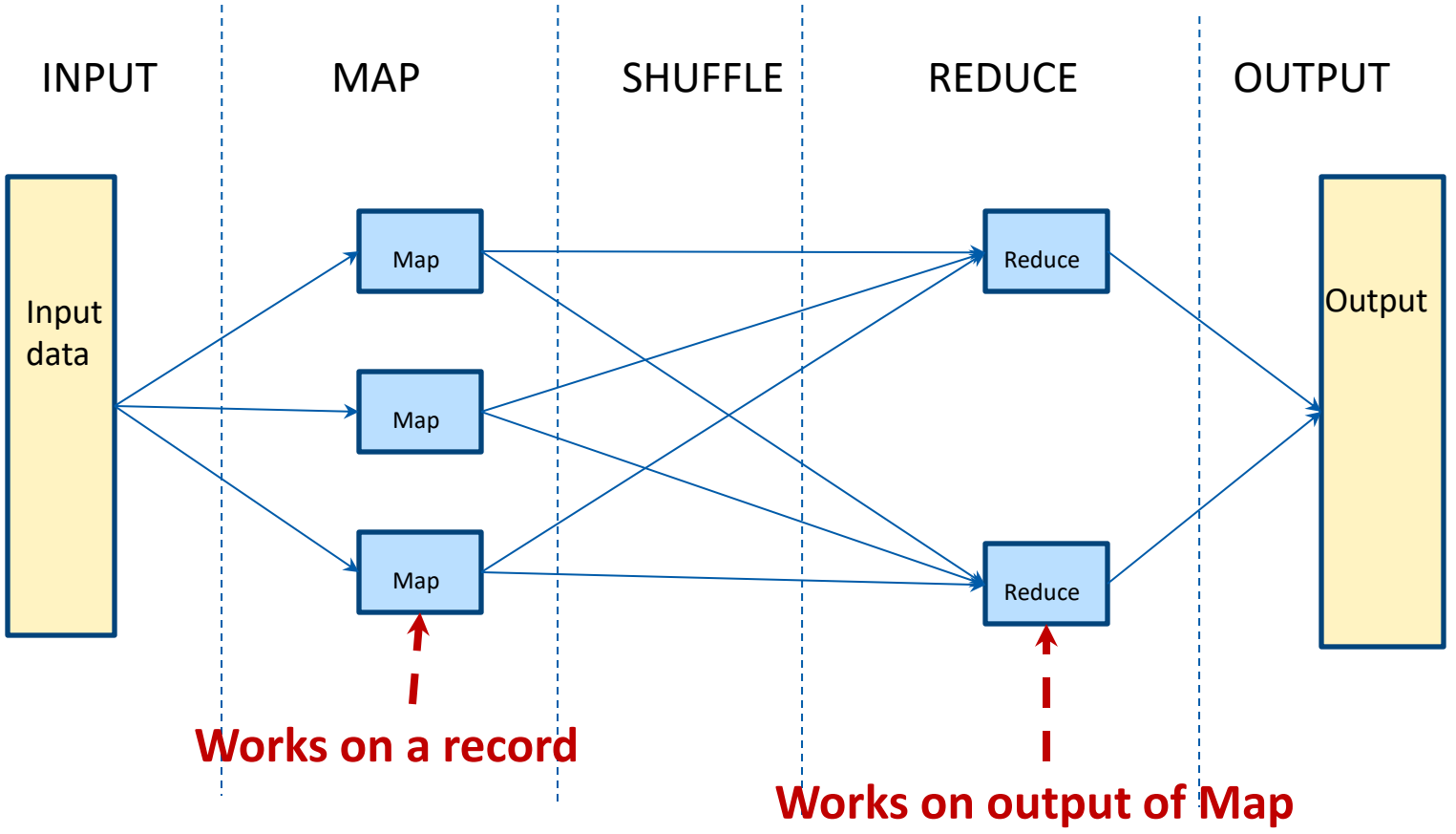
MapReduce: Combiner

- **Combiner:** do local aggregation/combine task at mapper

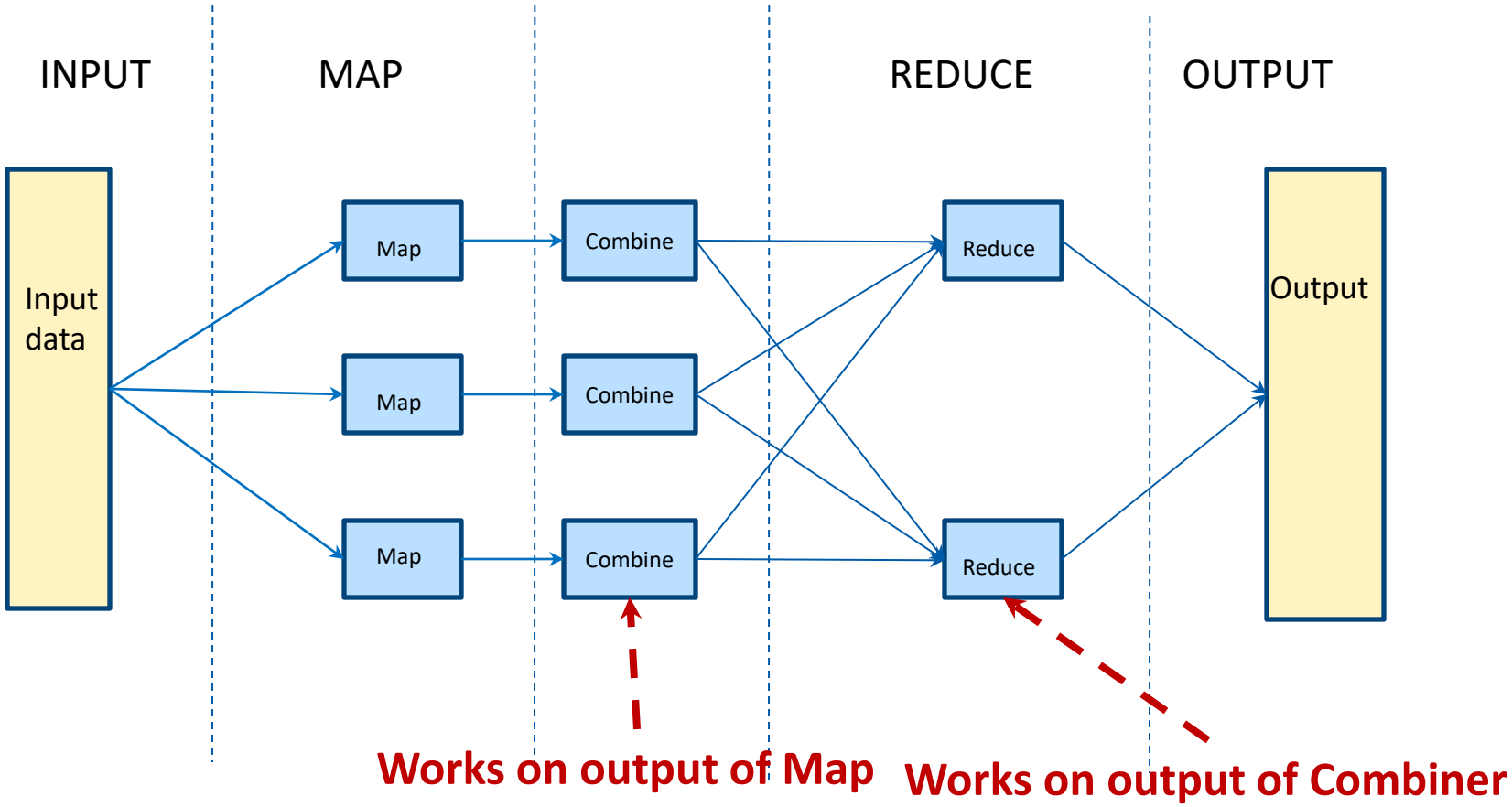


- **Q: What are the benefits of using combiner:**
  - Reduce memory/disk requirement of Map tasks
  - Reduce network traffic
- **Q: Can we remove the reduce function?**
  - No, reducer still needs to process records with same key but from *different mappers*
- **Q: How would you implement combiner?**
  - It is the same as Reducer!

MapReduce Overview



MapReduce Overview



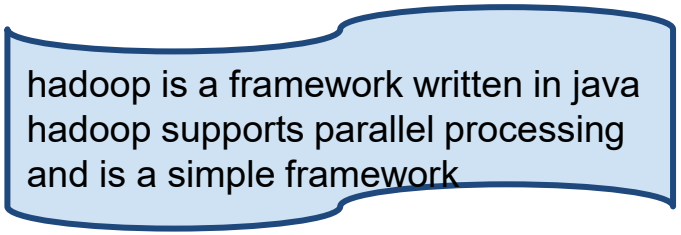
MapReduce Summary

- Mapper, Reducer and Combiner act on <key, value> pairs
- Map function gets one record at a time as an input
- Combiner (if present) works on output of map
- Reducer works on output of map (or combiner, if present)
- Combiner can be thought of local-reducer
  - Reduces output of maps that are executed on same node

Program (Demo)

Task: Given a text file, generate a list of words with the number of times each of them appear in the file

Input: Plain text file



Expected Output:

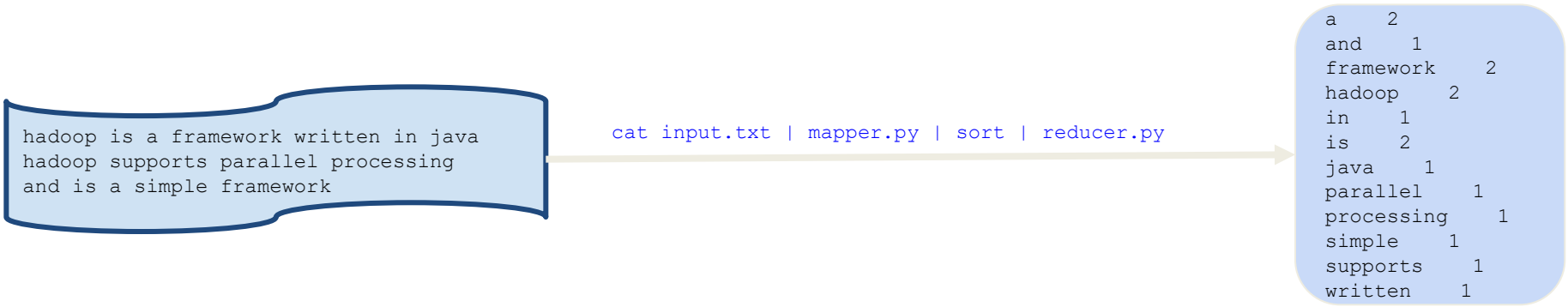
- <*word, frequency*> pairs for all words in the file

<hadoop, 2> <is, 2> <a , 2> <java , 1>	<framework , 2> <written , 1> <in , 1> <and,1>	<supports , 1> <parallel , 1> <processing. , 1> <simple,1>

Mimicking the Hadoop Flow

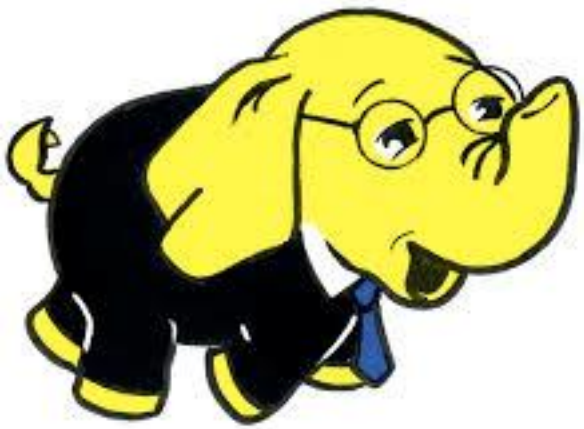
- Create files “mapper.py” for Map and “reducer.py” for Reduce
- Mimic Hadoop using the Linux pipe (|)

```
cat input.txt | mapper.py | sort | reducer.py
```



Further Reading

- Hadoop: The Definitive Guide: Tom White
- http://developer.yahoo.com/hadoop/tutorial/
- http://www.cloudera.com/content/cloudera-content/cloudera-docs/HadoopTutorial/CDH4/Hadoop-Tutorial.html



Thank You